

Computer Programs in Physics

DUGKS-GPU: An efficient parallel GPU code for 3D turbulent flow simulations using Discrete Unified Gas Kinetic Scheme ☆,☆☆

Kairzhan Karzhaubayev^a, Lian-Ping Wang^{a,*}, Dauren Zhakebayev^b^a Guangdong-Hong Kong-Macao Joint Laboratory for Data-Driven Fluid Mechanics and Engineering Applications, Southern University of Science and Technology, Shenzhen, Guangdong province, China^b National Engineering Academy of Republic of Kazakhstan, International Engineering Technological University, Almaty, Kazakhstan

ARTICLE INFO

Keywords:

Gas-kinetic scheme
DUGKS
CUDA
GPU
Taylor-Green
Turbulent channel flow

ABSTRACT

This paper presents a parallel implementation of the Discrete Unified Gas Kinetic Scheme (DUGKS) on the GPU system using the CUDA Fortran and CUDA C++ programming languages. Firstly, we conducted an extensive revision of our original CPU-based code, resulting in a threefold decrease in memory usage. This new implementation is also paired with a novel approach to compute cell face flux using trilinear interpolation. It is shown analytically that the interpolation-based approach to flux calculation is more accurate compared to the one used in the original DUGKS. The initial simulation results using this new approach suggest that trilinear interpolation can reduce numerical errors on a coarse mesh. For example, in the case of the decaying Taylor-Green vortex flow at a 128^3 mesh resolution, the relative numerical error in the energy dissipation rate at $t^* = 2$, using the spectral simulation result as the benchmark, is approximately 30% lower than that of the original implementation. The improved GPU DUGKS method is applied to laminar and turbulent flows in periodic and wall-bounded boundary configurations. A performance comparison of the GPU implementation is also presented and compared to the previous CPU implementation. A maximum speedup of 7.64x was achieved on a desktop-level GPU compared to a 32-core CPU. The strong scaling test, conducted on an eight-GPU node, demonstrated the efficient utilization of available multiple GPU resources by the code.

Program summary

Program Title: DUGKS-GPU

CPC Library link to program files: <https://doi.org/10.17632/yykv5s9g2n.1>Developer's repository link: <https://github.com/kairzhan/DUGKS-GPU>Code Ocean capsule: <https://codeocean.com/capsule/3b1e4f74-cdd3-4781-8923-8514d5923dfb/>

Licensing provisions: GNU General Public License 3

Programming language: CUDA C++, CUDA Fortran

Supplementary material:

Nature of problem: DUGKS-GPU is an advanced numerical code designed to accelerate the simulation of turbulent fluid flows through the application of the kinetic method known as the Discrete Unified Gas Kinetic Scheme (DUGKS). This computational tool comprises a CUDA Fortran version, tailored for a single GPU, as well as a CUDA C++ version developed for multi-GPU platforms.

Solution method: The code employs a second-order finite-volume discretization of the discrete velocity Boltzmann equation with the BGK collision model. It can be used to simulate small to medium-scale problems on modern multi-GPU platforms with CUDA architecture.

☆ The review of this paper was arranged by Prof. David W. Walker.

☆☆ This paper and its associated computer program are available via the Computer Physics Communications homepage on ScienceDirect (<http://www.sciencedirect.com/science/journal/00104655>).

* Corresponding author.

E-mail address: wanglp@sustech.edu.cn (L.-P. Wang).

1. Introduction

Graphics Processing Unit (GPU) systems have become an integral part of modern computational fluid dynamics (CFD) [1]. The first GPU from NVIDIA, capable of such general-purpose computations (GPGPU), was the G80 chip. Introduced in 2006, the G80 was later combined with Compute Unified Device Architecture [2] programming methodology, allowing the broad audience to use many-core systems for solving their engineering and scientific problems. The G80 featured 16 Streaming Multiprocessors (SM), each SM contained 8 CUDA Cores, giving in total 128 cores per GPU chip. On a debut, 128 of such cores running at 612 MHz frequency gained a lot of attention from the high-performance computing community. The relatively recent consumer-grade GPU 4090 debuted in 2022 is based on an Ada Lovelace architecture and features 128 SMs with 128 cores in each SM, resulting in 16384 CUDA cores running at 2235 MHz. The declared peak performance of 82.58 TFLOPS sounds very appealing for desktop-level teraflops workstations.

Aside from hardware and parallelization perspective, the second essential component of numerical simulations is the numerical method. Popular non-meshfree methods include Finite-Difference (FD)/Finite-Volume (FV) methods and methods based on the discretization of the Boltzmann equation, so-called mesoscopic methods. Mesoscopic methods rely on a kinetic description of fluid particles whose integral moments can reproduce the known macroscopic partial differential equations. Examples of mesoscopic methods are Lattice-Boltzmann Method (LBM) [3], Gas-Kinetic Scheme (GKS) [4] and recently developed Discrete Unified Gas Kinetic Scheme (DUGKS) [5]. The common unifying features of mesoscopic methods are their data locality and the explicit form of the time-marching algorithm. As a result, an efficient parallel implementation can be formulated on modern multicore parallel systems [6].

One of the first applications of CUDA for LBM was conducted by Tölke [7] in 2010 for NVIDIA 8800 Ultra GPU. Special care was taken to improve memory bandwidth using shared memory for data transfers between the main memory and the GPU computational kernels. About one order of magnitude speedup could be achieved when compared to desktop CPU workstation [7].

Soon after, an alternative data transfer scheme during the streaming step in LBM was implemented and tested by Obrecht et al. [8]. Their results indicate that a good performance can be achieved in the GPU implementation of LBM without the usage of shared memory, in addition, the analysis of GPU assembler code indicated that more complex collision models, such as the multiple-relaxation-time models in LBM, do not contribute much to overall time cost since LBM is still a more memory-bound algorithm.

In order to introduce GPU LBM implementation for multi-physics applications, Habich et al. [9] introduced a massively parallel framework known as widely applicable Lattice Boltzmann from Erlangen (waL-Berla). They compared the performances of NVIDIA GPUs, AMD GPUs, and Intel Xeon CPUs, and demonstrated that a good performance was attained on both CUDA and OpenCL LBM implementations on GPUs. However, it was found that activation of memory error correction codes (ECC) can significantly reduce the overall GPU performance.

In the LBM community, it is known that the LBM has a performance advantage on parallel systems because the rule “non-linearity is local, non-locality is linear” [3] describes LBM algorithm well and encourages the usage of parallel multi-core systems. Non-locality of the non-linear terms is common for FD/FV methods. An example of FD code for direct-numerical simulations of atmospheric flows is the MicroHH [10] - fourth order code with CPU and GPU parallelizations, where the single GPU parallelized version achieves only around 1.59x performance gain compared to 32-core CPU versions, while Bailey et al. [11] reported the speedup of 27.7 over 4-core CPU simulation for an LBM code.

Solving essentially the same model Boltzmann equation as in LBM but incorporating the finite-volume formulation when treating the lin-

ear advection term, the relatively new kinetic method DUGKS, has already gained much interest from the CFD community. Recent applications and extensions of DUGKS include both incompressible and compressible turbulent flows [12,13], thermal flows [14,15], particle-laden flows [16,17], and flows in non-continuum regimes [18]. Guo and Xu [19] provided a review of recent progress in DUGKS. DUGKS requires local interpolations when updating the distribution functions at the cell interface, but these interpolations remain local. The time-marching algorithm can be made explicit by applying linear transformations similar to those used in the Lattice-Boltzmann Method (LBM). Thus, an efficient parallel application of DUGKS on the GPU systems can also be achieved and should be explored.

The first porting and usage of DUGKS on GPU systems known to us were reported by Shang et al. [20] and Liu et al. [21]. Shang et al. [20] applied the CUDA code for DUGKS simulations, however, details other than a comparison of the runtimes were not given. Liu et al. [21] performed a series of validation and benchmarking cases for laminar two- and three-dimensional continuum and rarefied flows on a GPU. They reported a speedup of 338x on V100 GPU over a single CPU thread. Their encouraging results motivated us to explore the applicability of DUGKS on a GPU system, primarily focusing on the simulation of turbulent flows. Specifically, we would like to address the following questions:

1. How to reduce the memory usage of a DUGKS code by utilizing different memory levels of GPUs?
2. What are the resulting speedups when simulating turbulent flows?
3. Can alternative interpolations be applied when updating the distribution functions at the cell interface for local flux computation, and how does this change affect the overall accuracy of DUGKS?

Since our previous DUGKS code [22] used several auxiliary arrays to store intermediate data, resulting in large global memory usage, we shall resolve this issue by storing all auxiliary data in the local memory and registers of the GPU. This technique could be explored on both CPU and GPUs. A study of the second question above is required to further extend the applicability of DUGKS for large-scale turbulent flows. In addition, an alternative approach to flux computation is presented, and its numerical results were compared to those from the original flux-computation technique.

The source code for the developed GPU program has been released under the GPL license, with the aim of fostering wider adoption of the DUGKS method and contributing to the collection of available mesoscopic simulation codes. As of our knowledge, there is currently only one open-source DUGKS implementation known as dugksFoam [23]. DugksFoam is built on the OpenFOAM framework and utilizes the MPI library for multi-node CPU-based parallelization. In contrast, our approach is tailored for execution on multi-GPU configurations. The structured mesh employed in this work also contributes to enhanced performance, as stencil operations become more efficient compared to similar operations on an unstructured mesh.

The remaining parts of the paper are structured as follows. In Section 2, the details of the numerical method are presented, and Section 3 explains the implementation details for GPU, followed by a presentation of validation cases in Section 4. Discussion of the code's performance in single and multi-GPU configurations, as well as a comparison with computational speedup found in the literature, is presented in Section 5. The final section provides a summary of the work and prospects for future extensions.

2. Numerical method

Derivation of the DUGKS numerical procedure starts with the evolution equation for the discrete distribution function f_α [5]:

$$\frac{\partial f_\alpha}{\partial t} + \xi_\alpha \cdot \nabla f_\alpha = \Omega_\alpha^* = -\frac{f_\alpha - f_\alpha^{eq}}{\tau} + \Phi_\alpha \quad (1)$$

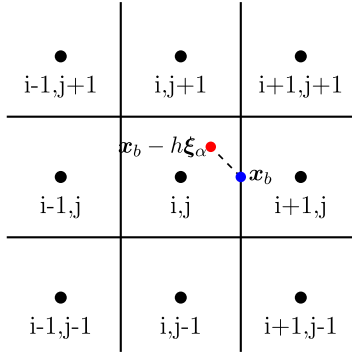


Fig. 1. Cell configuration used in the FV formulation where for simplicity, only the first two directions were shown. Here a particular direction of the discrete velocity $\xi = \{1, -1\}$ is considered for a cell-interface distribution at the half-time step, a particle from the red node at t_n would approach the cell face center at the blue node at $t_{n+0.5}$. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

where α indicates a discrete velocity direction, τ is the relaxation rate, Ω^* denotes the collision term with the external forcing term included, and, for a continuum flow, the mesoscopic forcing term Φ can be approximated as [24]

$$\Phi_\alpha \approx \frac{\mathbf{F} \cdot (\xi_\alpha - \mathbf{u})}{\rho RT} f_\alpha^{eq} = \phi_\alpha f_\alpha^{eq}. \quad (2)$$

For incompressible flow, the equilibrium distribution function is typically expanded as

$$f_\alpha^{eq} = \rho W_\alpha \left(1 + \frac{\xi_\alpha \cdot \mathbf{u}}{RT} + \frac{1}{2} \frac{(\xi_\alpha \cdot \mathbf{u})^2}{(RT)^2} - \frac{\mathbf{u} \cdot \mathbf{u}}{2RT} \right) \quad (3)$$

where the weight coefficients are $W_0 = 1/3$, $W_{1-6} = 1/18$, $W_{7-18} = 1/36$, ρ is the local fluid density, $\mathbf{u}$ is the local fluid velocity, $\mathbf{F}$ is the macroscopic body force per unit volume and $RT = c_s^2$, where c_s is the model speed of sound. In this work, the D3Q19 [22] discrete velocity model was considered.

The macroscopic quantities are defined as

$$\rho = \sum f_\alpha, \quad \rho \mathbf{u} = \sum \xi_\alpha f_\alpha, \quad p = \rho RT. \quad (4)$$

Discretization in spatial coordinates is performed using the finite volume (FV) formulation, which converts the advection term to the flux computation on the cell interfaces. As for time integration, the trapezoidal rule is used for the collision term, and the midpoint rule is used for the resulting flux terms. A typical mesh arrangement for the case of orthogonal uniform mesh is shown in Fig. 1, where for a given cell at (i, j, k) , the discrete evolution equation takes the form

$$f_{\alpha;i,j,k}^{n+1} - f_{\alpha;i,j,k}^n + \frac{\Delta t}{V_{i,j,k}} \mathcal{F}_{\alpha;i,j,k}^{n+1/2} = \frac{\Delta t}{2} \left[\Omega_{\alpha;i,j,k}^{*n+1} + \Omega_{\alpha;i,j,k}^{*n} \right], \quad (5)$$

where the flux term is expressed as

$$\mathcal{F}_{\alpha;i,j,k}^{n+1/2} = \sum_{faces} \xi_\alpha \cdot \mathbf{n}_{face} f_{\alpha,face}^{n+1/2} \Delta S_{face}, \quad (6)$$

where $face$ indicates particular boundary face of the FV cell, $\mathbf{n}_{face}$ is the outward unit normal of the boundary face and ΔS_{face} represents surface area of that boundary face.

In the equations above, $f_{\alpha;i,j,k}$ corresponds to the value of f_α averaged over the volume of (i, j, k) cell.

Implicitness in the RHS of Eq. (5) is removed by introducing two new distribution functions:

$$\bar{f}_\alpha = f_\alpha - \frac{\Delta t}{2} \Omega_\alpha^* \quad (7)$$

$$\bar{f}_\alpha^+ = f_\alpha + \frac{\Delta t}{2} \Omega_\alpha^*. \quad (8)$$

Then the evolution equation, Eq. (5), reduces to

$$\bar{f}_{\alpha;i,j,k}^{n+1} = \bar{f}_{\alpha;i,j,k}^{n+1/2} - \frac{\Delta t}{V_{i,j,k}} \mathcal{F}_{\alpha;i,j,k}^{n+1/2}. \quad (9)$$

To evaluate the flux term at the half timestep $t_{n+0.5} = t_n + h$, where $h = 0.5\Delta t$, the distribution function at the cell face center ($\mathbf{x}_b$) at $t_{n+0.5}$ is obtained by integrating Eq. (1) along the characteristic line from t_n to $t_{n+0.5}$, yielding the following expression

$$f_\alpha(\mathbf{x}_b, t_n + h) = f_\alpha(\mathbf{x}_b - h\xi_\alpha, t_n) + \frac{h}{2} \left[\Omega_\alpha^*(\mathbf{x}_b, t_n + h) + \Omega_\alpha^*(\mathbf{x}_b - h\xi_\alpha, t_n) \right]. \quad (10)$$

Again, the following linear transformations are introduced to remove the implicit terms:

$$\bar{f}_\alpha = f_\alpha - \frac{h}{2} \Omega_\alpha^* \quad (11)$$

$$\bar{f}_\alpha^+ = f_\alpha + \frac{h}{2} \Omega_\alpha^*. \quad (12)$$

Eq. (10) then takes the form

$$\bar{f}_\alpha(\mathbf{x}_b, t_n + h) = \bar{f}_\alpha^+(\mathbf{x}_b - h\xi_\alpha, t_n). \quad (13)$$

The RHS of Eq. (13) requires evaluation of $\bar{f}_\alpha^+$ at t_n at an off-grid location $\mathbf{x}_b - h\xi_\alpha$. There are different possibilities. In Guo et al.'s original paper [5], a Taylor expansion around the cell interface $\mathbf{x}_b$ was used

$$\bar{f}_\alpha^+(\mathbf{x}_b - h\xi_\alpha, t_n) = \bar{f}_\alpha^+(\mathbf{x}_b, t_n) - h\xi_\alpha \cdot \nabla \bar{f}_\alpha^+(\mathbf{x}_b, t_n), \quad (14)$$

which then requires evaluation of $\bar{f}_\alpha^+$ and its gradients at the cell interface $\mathbf{x}_b$. Guo et al. [5] computes $\bar{f}_\alpha^+(\mathbf{x}_b, t_n)$ by taking the average value of $\bar{f}_\alpha^+$ at two adjacent cell centers, namely, (i, j, k) and $(i + 1, j, k)$ for the example shown in Fig. 1 with $\mathbf{x}_b = \mathbf{x}_{i,j,k} + 0.5\Delta x \mathbf{e}_x$, where $\mathbf{e}_x$ is the unit vector along the x direction. Using the introduced transformations, we can show that $\bar{f}_\alpha^+$ at the cell center can be computed as

$$\bar{f}_{\alpha;i,j,k}^+ = \frac{2\tau - h}{2\tau + \Delta t} \bar{f}_{\alpha;i,j,k} + \left(\frac{3h}{2\tau + \Delta t} + \frac{3h\tau}{2\tau + \Delta t} \phi_\alpha \right) f_{\alpha;i,j,k}^{eq}. \quad (15)$$

When computing the gradients, the normal gradient is evaluated by central differencing, namely, $[\bar{f}_\alpha^+(\mathbf{x}_{i+1,j,k}, t_n) - \bar{f}_\alpha^+(\mathbf{x}_{i,j,k}, t_n)] / \Delta x$ for the example in Fig. 1 on a uniform mesh. For the tangential gradients, again the central differencing was used, e.g., using the two interface values at $\mathbf{x}_b \pm \Delta y \mathbf{e}_y$ for the y -gradient, where $\mathbf{e}_y$ is the unit vector in the y direction. Namely, for the example shown in Fig. 1,

$$\begin{aligned} \frac{\partial \bar{f}_\alpha^+(\mathbf{x}_b, t_n)}{\partial y} &= \frac{\bar{f}_\alpha^+(\mathbf{x}_b + \Delta y \mathbf{e}_y, t_n) - \bar{f}_\alpha^+(\mathbf{x}_b - \Delta y \mathbf{e}_y, t_n)}{2\Delta y} \\ &\approx \frac{0.5 \left(\bar{f}_{\alpha;i,j+1,k}^{+,n} + \bar{f}_{\alpha;i+1,j+1,k}^{+,n} \right) - 0.5 \left(\bar{f}_{\alpha;i,j-1,k}^{+,n} + \bar{f}_{\alpha;i+1,j-1,k}^{+,n} \right)}{2\Delta y} \end{aligned} \quad (16)$$

To summarize, the computation of Eq. (14) in Guo et al. [5] makes use of a stencil containing a maximum of 6 cell-center node points in 2D, or a stencil containing a maximum of 10 cell-center node points in 3D. We shall refer to the above approach as the gradient-based interpolation.

Here, we propose a trilinear interpolation using only the nearest cell-center node points, which means a stencil containing a maximum of 4 cell-center node points in 2D, or a stencil containing a maximum of 8 cell-center node points in 3D. This stencil choice is more compact in space compared to Guo et al.'s approach (Fig. A.19). Observing from the Fig. 1 that the required value of $\bar{f}_\alpha^+(\mathbf{x}_b - h\xi_\alpha, t_n)$ is located at the location indicated by the red circle symbol, which is inside of the domain formed by the four cell-center nodes, (i, j) , $(i + 1, j)$, $(i, j + 1)$, $(i + 1, j + 1)$, in 2D, then its value can be directly interpolated from $\bar{f}_\alpha^+$ values at

these four cell-center node points. The trilinear interpolation has potential benefits in improving the locality of involved data. For example, the central difference approximation in the gradient-based interpolation, Eq. (14) at position $(i + 1/2, j, k)$, uses nodes (i, j, k) and $(i + 1, j, k)$ to approximate the x derivative, while nodes $(i, j + 1, k)$, $(i + 1, j + 1, k)$, and $(i, j - 1, k)$, $(i + 1, j - 1, k)$ are used to estimate the y gradient as indicated by Eq. (16). Hence, the y gradient calculation does not use data from the closest nodes (i, j, k) and $(i + 1, j, k)$. On the other hand, the trilinear interpolation always uses the values from the closest cell-center nodes to perform the interpolation.

In Appendix A, we theoretically compare the trilinear interpolation with the gradient-based interpolation to show that the more compact stencil used in the trilinear interpolation indeed results in a more accurate approximation in computing $\bar{f}_\alpha^+(\mathbf{x}_b - h\xi_\alpha, t_n)$.

In addition, the trilinear interpolation involves a smaller number of floating-point operations. For example, for three-dimensional CPU-based simulations, we have compared the run times of two DUGKS codes, one uses gradient-based interpolation, and the other uses trilinear interpolation, while other parts of the codes are identical. We found that the trilinear interpolation code is roughly 21% faster when compared to the DUGKS code using gradient-based interpolation.

For the above reasons, we believe that the trilinear interpolation is better than the gradient-based interpolation. Therefore, in our GPU implementation, the trilinear interpolation is adopted.

For convenience, $\bar{f}$ is used as the main evolving variable in DUGKS. The sequence of computations of each DUGKS time step is shown in Algorithm 1. The expressions of Eqns. (19) and (20) originate from the fact that all the modified distribution functions $\bar{f}$, $\bar{f}^+$, $\bar{f}^-$, $\bar{f}^+$ are linear combinations of f and f^{eq} .

Algorithm 1: The algorithm for each DUGKS step

- 1 Using the value of $\bar{f}_{\alpha,i,j,k}^n$, compute $\bar{f}_{\alpha,i,j,k}^{+,n}$ using Eq. (15);
- 2 Using the trilinear interpolation, approximate the value of $\bar{f}_\alpha^+(\mathbf{x}_b - h\xi_\alpha, t_n)$, which then yields $\bar{f}_\alpha(\mathbf{x}_b, t_n + h)$ following Eq. (13);
- 3 Compute the macroscopic quantities $\rho(\mathbf{x}_b, t_n + h)$, $\mathbf{u}(\mathbf{x}_b, t_n + h)$ at cell faces from the moments of $\bar{f}_\alpha(\mathbf{x}_b, t_n + h)$:

$$\rho = \sum_\alpha \bar{f}_\alpha \quad (17)$$

$$\rho \mathbf{u} = \sum_\alpha \xi_\alpha \bar{f}_\alpha + \frac{h}{2} \mathbf{F}; \quad (18)$$

- 4 From the computed macroscopic quantities at the cell faces, compute $\bar{f}_\alpha^{eq}(\mathbf{x}_b, t_n + h)$;
- 5 Recover the value of the original distribution function at the cell faces, $f_\alpha(\mathbf{x}_b, t_n + h)$, using

$$f_\alpha(\mathbf{x}_b, t_n + h) = \frac{2\tau}{2\tau + h} \bar{f}_\alpha(\mathbf{x}_b, t_n + h) + \left(\frac{h}{2\tau + h} + \frac{\tau h}{2\tau + h} \phi_\alpha \right) \bar{f}_\alpha^{eq}(\mathbf{x}_b, t_n + h) \quad (19)$$

then compute the required fluxes at the half timestep $t_{n+1/2} = t_n + h$;

- 6 Compute $\bar{f}_{\alpha,i,j,k}^{+,n}$ using
- $$\bar{f}_{\alpha,i,j,k}^{+,n} = \frac{4}{3} \bar{f}_{\alpha,i,j,k}^{+,n} - \frac{1}{3} \bar{f}_{\alpha,i,j,k}^n; \quad (20)$$
- 7 Evolve the distribution function $\bar{f}_{\alpha,i,j,k}$ to $t_n + \Delta t$ using Eq. (9).
-

3. Implementation on a GPU

CUDA GPU contains a fixed number of SMs, and each SM can execute a logical group of threads called a *block*. Threads in a block are

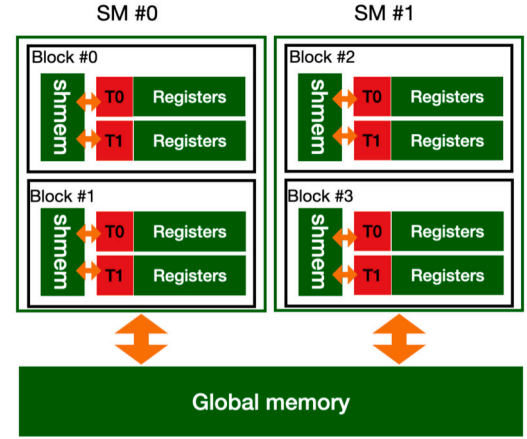


Fig. 2. Simplified memory hierarchy used in CUDA. Here, it is assumed that the GPU contains two SMs, and each SM contains two blocks of threads with two threads (T0 and T1) per block. Each thread has access to three types of memory storage: per-thread private memory (registers), per-block shared memory, and global device memory.

further partitioned into so-called *warps*, and warps in SM perform the same instructions in SM simultaneously.

Considering a large number of available CUDA cores, the standard way of assigning tasks to CUDA cores is to invoke CUDA kernels for each node data value. Previously our implementation assigned global arrays to store fluxes at the cell faces, which increased memory usage for DUGKS. The implementation presented here used additional memory levels to reduce global memory usage.

From programmers' perspective, CUDA GPUs provide several layers of memory (Fig. 2). First, there is the so-called global or *device memory*, usually large (up to 80 GB in modern datacenter level GPUs). This memory is accessible by all threads of the GPU. While this memory has a wide data transfer bus, it is considered slow compared to the *shared memory* assigned to each SM. All threads which are grouped into blocks and performed on SM can use this memory collectively. The last memory type is the local *registers*. Registers are privately assigned to each running thread and are used for intermediate calculations.

A typical work-sharing procedure for mesh-based calculations is shown for two-dimensional configuration in Fig. 3. In this example, the two-dimensional mesh of 32×32 cells is divided into eight blocks, where each block contains 128 cells in a 16×8 configuration. If the GPU device contains eight or more SMs, all these blocks can be launched and executed on a GPU simultaneously. To improve the locality and the availability of the data, all threads in a block copy block-associated data to the shared memory of the block. This is illustrated in Fig. 4, where block data and halo layers represent the typical content of per-block shared memory. The pseudo-code version of this copy procedure is also shown in Algorithm 2. Such an approach is sensitive to the block size and the size of the shared memory in the SM. For example, for a block size of $16 \times 8 \times 1$, storage of block-related data with halo layers for all values of the distribution function in the D3Q19 model requires $(16 + 2) \times (8 + 2) \times (1 + 2) \times 19 \times 4 = 41040$ bytes of memory in the single precision mode, or twice this memory in the double precision mode. In our previous CPU based implementation [22], six main and auxiliary four-dimensional arrays were used to store information of particle distributions. The main arrays include arrays $\text{fb}(0:\text{npop}-1, \text{nx}, \text{ny}, \text{nz})$ and $\text{f}(0:\text{npop}-1, \text{nx}, \text{ny}, \text{nz})$ to store values of $\bar{f}_\alpha^+$ and $\bar{f}_\alpha$, respectively, where $\text{npop} = 19$ and the triple $\text{nx}, \text{ny}, \text{nz}$ defines number of cells in three Cartesian directions. Four additional arrays $\text{fix}(0:\text{npop}-1, 0:\text{nx}, 0:\text{ny}+1, 0:\text{nz}+1)$, $\text{fiy}(0:\text{npop}-1, 0:\text{nx}+1, 0:\text{ny}, 0:\text{nz}+1)$, $\text{fiz}(0:\text{npop}-1, 0:\text{nx}+1, 0:\text{ny}+1, 0:\text{nz})$ and $\text{fi}(0:\text{npop}-1, 0:\text{nx}+1, 0:\text{ny}+1, 0:\text{nz}+1)$ were used to store the face averaged values of $\bar{f}_\alpha^+$ and $\bar{f}_\alpha$. In the current GPU code, only two four-dimensional arrays, $\bar{f}_\alpha^+$ and $\bar{f}_\alpha$, are needed.

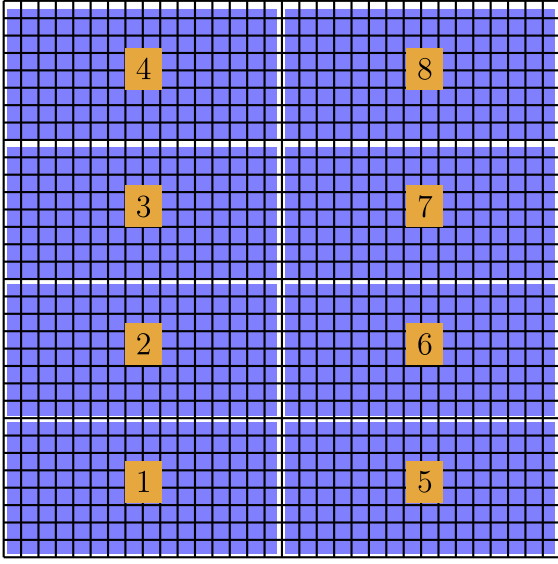


Fig. 3. 32×32 mesh of cell. This domain is divided into a two-dimensional grid of eight blocks and each block is assigned to be executed on a single SM. The labels indicate separate blocks.

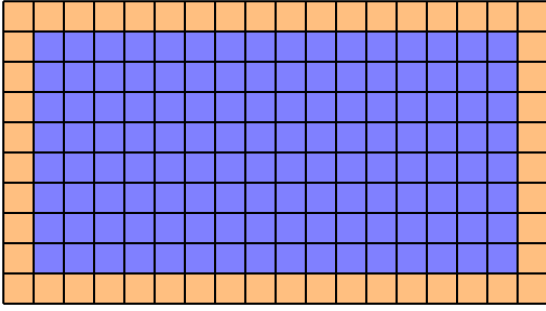


Fig. 4. Each block contains 16×8 mesh cells (blue color) plus the additional number of halo cells (orange). For this example, each CUDA block processes 128 ($= 16 \times 8$) data cells.

The intermediate values required to perform time marching from t^n to t^{n+1} are stored in block shared memory and in local per-thread registers.

Algorithm 2: Shared memory usage by the group of threads in a block

```

1 // first, acquire corresponding cell position;
2  $i = \text{threadIdx}\%x + (\text{blockIdx}\%x - 1) * \text{blockDim}\%x$ ;
3  $j = \text{threadIdx}\%y + (\text{blockIdx}\%y - 1) * \text{blockDim}\%y$ ;
4  $k = \text{threadIdx}\%z + (\text{blockIdx}\%z - 1) * \text{blockDim}\%z$ ;
5 // then find corresponding location in shared memory;
6  $i_0 = \text{threadIdx}\%x$ ;
7  $j_0 = \text{threadIdx}\%y$ ;
8  $k_0 = \text{threadIdx}\%z$ ;
9 // each thread copies single cell data at position  $(i, j, k)$  to current
  block's shared memory;
10  $\bar{s}_{\alpha; i_0, j_0, k_0} \leftarrow \bar{f}_{\alpha; i, j, k}^+$ ;
11 // use the array  $\bar{s}$  instead of  $\bar{f}^+$  for interpolation purposes at step #2 of
  Algorithm 1;
```

Usage of the shared memory allowed to reduce global memory usage and more efficiently perform flux calculations using the trilinear interpolation procedure. Other than the target platform (GPU instead of CPU) and interpolation approach for face flux calculations, the DUGKS methodology we use is identical to the one presented in [22].

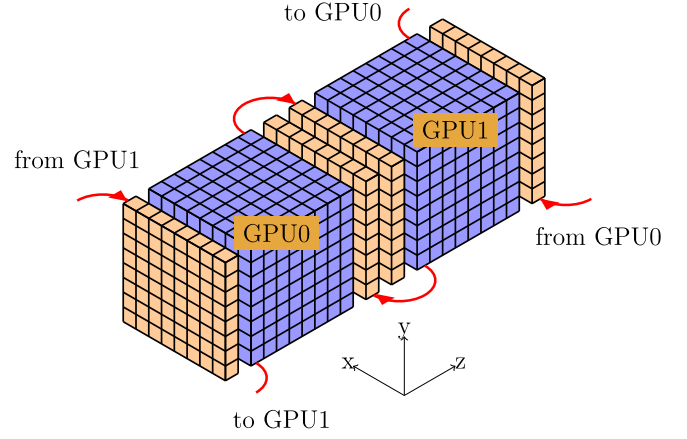


Fig. 5. Data exchange occurs through halo layers in one-dimensional domain decomposition. Here, periodic boundary condition in the z-direction is assumed, and only two subdomains are illustrated. The red arrows depict the direction of data transfer.

Table 1

Parameters used in the Taylor-Green vortex flow.

V_0	L	N	ν	Re	CFL
0.1018	128	128/256	$U_0 N / 600\pi$	1880	0.5

In addition to the application of DUGKS on a CUDA platform, the CUDA C version of the code was extended to a multi-GPU configuration using CUDA-aware MPI from the NVIDIA HPC software kit. The multi-GPU parallelization is based on a standard halo-layer approach, where halo-layers are employed in the z-direction (refer to Fig. 5). This one-dimensional domain decomposition, also known as slab decomposition, is considered highly efficient for a small number of subdomains. The results of the strong-scaling test are presented in Section 5 of this work.

4. Validation

4.1. Decaying Taylor-Green vortex flow

Direct numerical simulation of the three-dimensional Taylor-Green vortex flow is a popular test of the accuracy and the performance of a numerical method. In this test case, energy transfer from large to small scales is followed by the decay phase similar to the decaying homogeneous turbulence. Initial flow conditions are given as

$$\begin{aligned}
 u &= V_0 \sin\left(\frac{2\pi x}{L}\right) \cos\left(\frac{2\pi y}{L}\right) \cos\left(\frac{2\pi z}{L}\right) \\
 v &= -V_0 \cos\left(\frac{2\pi x}{L}\right) \sin\left(\frac{2\pi y}{L}\right) \cos\left(\frac{2\pi z}{L}\right) \\
 w &= 0
 \end{aligned}$$

with periodic boundary conditions on all sides. The iterative approach by Mei et al. [25] was used to initialize the mesoscopic distribution function. Flow parameters such as characteristics velocity V_0 , domain size L , mesh resolution N , kinematic viscosity ν , and CFL number are summarized in Table 1. The Reynolds number corresponding to this configuration is $Re = 1880$. Unless stated otherwise, single-precision mode is used for numerical simulations in this and following cases.

Low-order statistics used for validation include kinetic energy and its dissipation rate:

$$\begin{aligned}
 k &= \int E(\kappa) d\kappa \\
 \epsilon &= 2\nu \langle s_{ij} s_{ij} \rangle, \text{ where } s_{ij} \equiv \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right),
 \end{aligned}$$

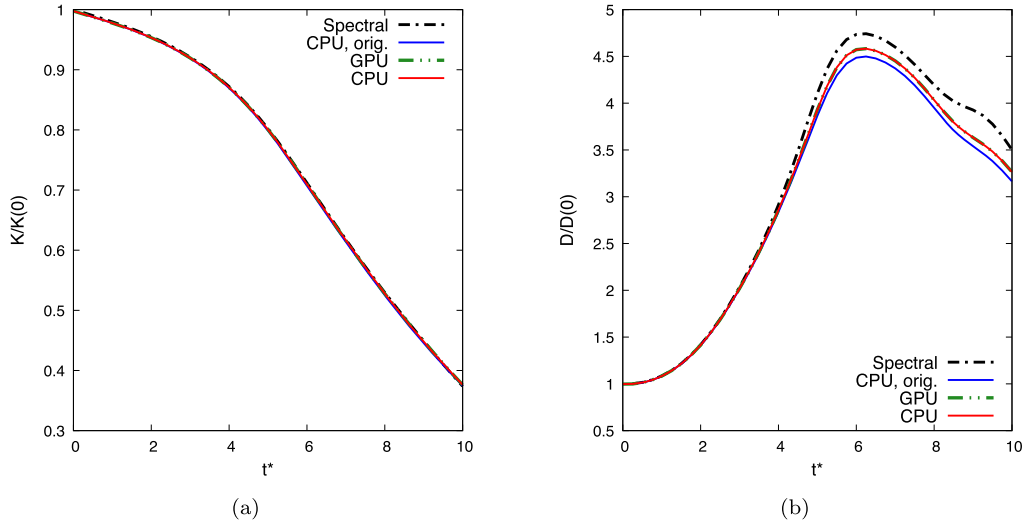


Fig. 6. Evolution of a) the kinetic energy and b) the energy dissipation rate for Taylor-Green vortex decay case using the uniform mesh with a resolution of 128^3 . In this and the following plots, “Spectral” indicates results from the pseudospectral simulations using the code [26], “CPU, orig.” results correspond to DUGKS with the original flux calculation approach, “CPU” and “GPU” correspond to DUGKS with interpolation-based flux calculation approach on CPU and GPU platforms respectively.

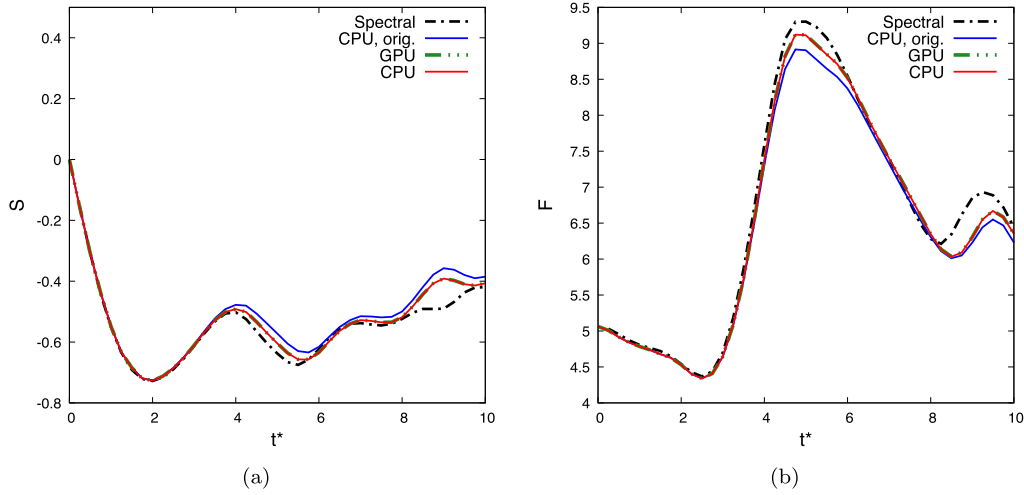


Fig. 7. Evolution of velocity derivative a) skewness and b) flatness for Taylor-Green vortex decay case using the uniform mesh with a resolution of 128^3 .

while for high-order statistics, velocity derivative skewness and flatness were considered:

$$S = \frac{\frac{1}{3}[(\frac{\partial u}{\partial x})^3 + (\frac{\partial v}{\partial y})^3 + (\frac{\partial w}{\partial z})^3]}{(\frac{1}{3}[(\frac{\partial u}{\partial x})^2 + (\frac{\partial v}{\partial y})^2 + (\frac{\partial w}{\partial z})^2])^{3/2}}$$

$$F = \frac{\frac{1}{3}[(\frac{\partial u}{\partial x})^4 + (\frac{\partial v}{\partial y})^4 + (\frac{\partial w}{\partial z})^4]}{(\frac{1}{3}[(\frac{\partial u}{\partial x})^2 + (\frac{\partial v}{\partial y})^2 + (\frac{\partial w}{\partial z})^2])^2}.$$

Simulation results are shown in Figs. 6 and 7 for a mesh resolution of 128^3 . For comparison purposes, results from our spectral code [26] are also shown, where the flow was adequately resolved as the resolution parameter $k_{max}\eta$ for mesh 128^3 varies between 2.65 and 1.936 for time interval from $\tilde{t} \equiv 2\pi V_0 t/L = 0$ to $\tilde{t} = 10$. The parameters $k_{max} = N/3$ and η respectively define the spectral truncation radius and the Kolmogorov length scale. The set of DUGKS results contains results from the DUGKS code using the original flux treatment and GPU and CPU-based results using the trilinear interpolation. Results using the trilinear interpolation performed on the GPU and CPU platforms

are identical, which indicates the correctness of numerical results obtained on a GPU platform. Compared to results using the original flux treatment in DUGKS, the trilinear-interpolation flux calculation yields better results of energy decay, velocity derivative skewness, and flatness when compared to the spectral simulation results.

The second simulation was performed using a finer mesh with a resolution of 256^3 , and respective results are shown in Figs. 8 and 9. Compared to the 128^3 mesh resolution, the 256^3 mesh resolution results using the original flux calculation are in an excellent agreement with results using the trilinear flux calculation. High-order statistics such as the velocity-derivative skewness and flatness from the two flux-calculation methods are almost identical, and only minor differences can be observed at $t \approx 5$ in both the skewness and flatness plots.

These results indicate that the proposed trilinear interpolation can be used for flow simulations in DUGKS and shows a better prediction of low and high-order statistics at relatively coarse mesh configurations. As the mesh resolution doubles, the two flux treatments, both of second-order accuracy, converge to yield almost identical results, similar to those from the spectral simulation.

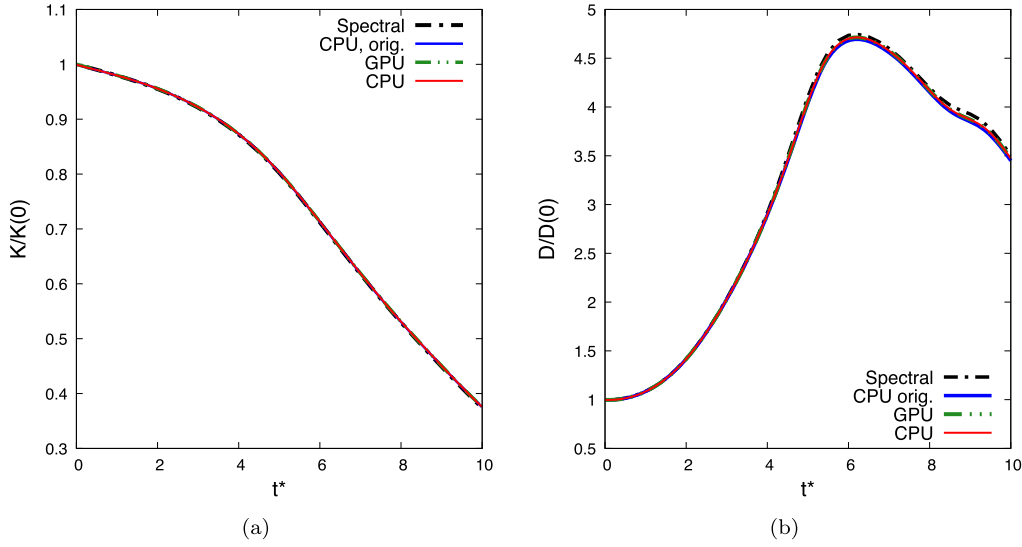


Fig. 8. Evolution of a) the kinetic energy and b) the energy dissipation rate for Taylor-Green vortex decay case using the uniform mesh with a resolution of 256^3 .

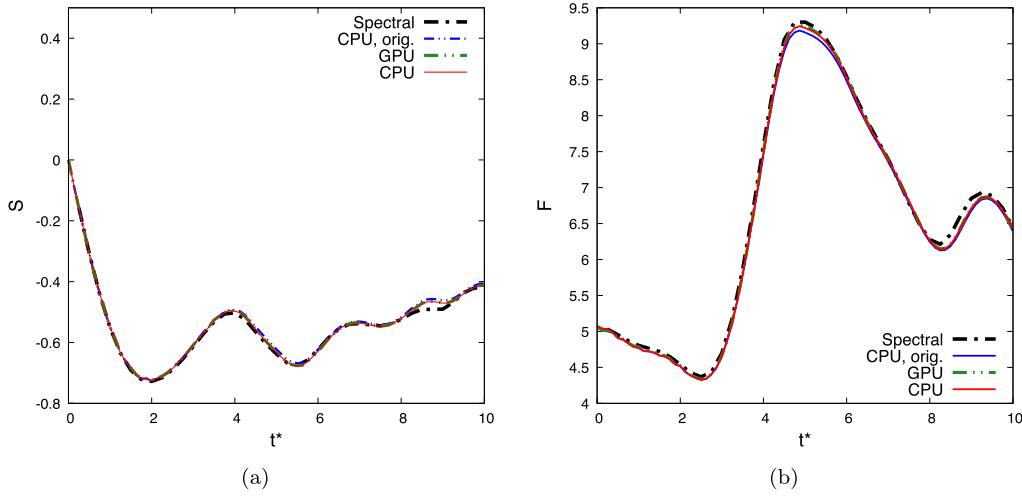


Fig. 9. Evolution of velocity derivative a) skewness and b) flatness for Taylor-Green vortex decay case using the uniform mesh with a resolution of 256^3 .

4.2. Decaying homogeneous isotropic turbulence

After validating the GPU implementation using the Taylor-Green vortex flow problem, the next problem considered was the decaying homogeneous isotropic turbulence (DHIT). For this case, Rogallo's procedure [27] was used to generate an initial divergence-free velocity field with the prescribed energy spectrum.

The initial energy spectrum is given by [12]

$$E_0(k) = Ak^4 e^{-0.14k^2},$$

with energy containing wavenumbers $k = [k_a, k_b] = [3, 8]$ and coefficient $A = 1.1474e - 2$. For the DHIT problem, the main parameter describing flow behavior is usually Taylor microscale-based Reynolds number Re_λ defined as

$$Re_\lambda = \frac{u_{rms} \lambda}{\nu},$$

where u_{rms} is Root-Mean-Square (RMS) velocity, λ is Taylor micro length scale, and ν is fluid kinematic viscosity. The remaining flow parameters such as L - domain size, N - mesh resolution, K_0 - initial kinetic energy, u_{rms} - initial RMS velocity fluctuations, ν - kinematic viscosity, Re_λ - initial Taylor microscale-based Reynolds number are summarized in Table 2.

Table 2

Parameters used in the DHIT case. Here DUGKS represents parameters for both original flux calculation method and to trilinear interpolation, spectral corresponds to pseudospectral simulation using the code from [26].

Method	L	N	K_0	u_{rms}	ν	Re_λ	CFL
DUGKS	256	256	1.5383e-3	0.0320	2.479e-2	26	0.7
Spectral	2π	256	0.9241	0.7849	1.4933e-2	26	0.4

Results from the GPU simulation using the trilinear interpolation are shown in Figs. 10 through 12, along with CPU results using the same trilinear interpolation and results from our spectral simulations. In this case, an excellent agreement is found between all three methods, which again validates the correctness and accuracy of the GPU implementation and of the improved interpolation scheme. Oscillatory behavior of the evolution of velocity-derivative skewness and flatness is usually attributed to the acoustic waves inherent in the kinetic method and high-order numerical errors [12].

4.3. Turbulent channel flow

The last problem is the turbulent channel flow, where a periodic flow in the two horizontal directions is confined in a vertical direction

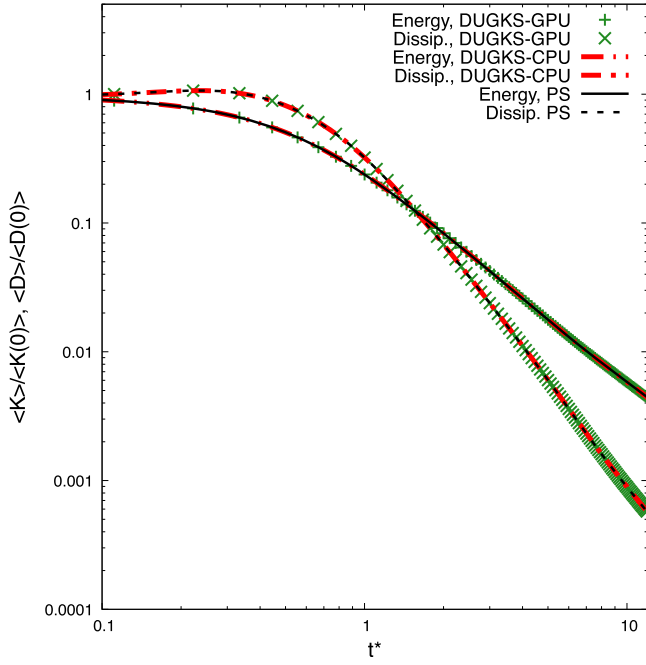


Fig. 10. Evolution of the kinetic energy and its dissipation rate for DHIT case with uniform mesh with a resolution of 256^3 . In this and the following plots, “PS” indicates the results from the pseudospectral simulations using the code [26], “DUGKS-CPU” and “DUGKS-GPU” correspond to DUGKS with interpolation-based flux calculation approach on CPU and GPU platforms, respectively.

with solid top and bottom walls, and a uniform and constant driving force is applied in the streamwise (x) direction. This case is simulated in a double-precision mode to eliminate any possible cumulative errors associated with long simulation runtimes. Compared to two previous cases, in the channel flow case, the non-uniform mesh with a resolution of $128 \times 128 \times 128$ was adopted with mesh refinement near the wall. The locations of the cell’s faces in a vertical direction are computed using

$$z_k = \frac{H}{\alpha} \tanh \left[\frac{1}{2} \eta (\ln(1 + \alpha) - \ln(1 - \alpha)) \right] + H, \quad (21)$$

where the parameter α represents the refinement scale and is equal to 0.85. Other simulation parameters are given in Table 3 and, aside from the mesh refinement procedure, are identical to those used in [22]. To

Table 3

Simulation parameters used for turbulent channel flow. $L_x \times L_y \times L_z$ represents domain size, H is channel half height, ν fluid kinematic viscosity, Re_τ is shear velocity based Reynolds number, u^* is a shear velocity, Δx_{min} is smallest cell height, and Δt is a timestep. Note that x is a wall normal direction and y is the streamwise direction.

$L_x \times L_y \times L_z$	H	ν	Re_τ	u^*	Δx_{min}	Δt
$2H \times 4\pi H \times 4\pi H/3$	30.86	0.0012	180	0.007	0.201	$2.84e-2$

break initial flow symmetry and bootstrap the turbulence, an unsteady force is applied at the beginning of the simulation, and details of this force are given in [22].

Reported flow statistics were averaged for a long period of more than 28 large eddy turnover times, after the flow reaches a statistically steady state. The average mean flow speed is found to be 15.74, which is consistent with the previous results for DUGKS [22]. The averaged velocity profile is shown in Fig. 13, followed by the averaged profiles of the Reynolds stress in Fig. 14 and the Root-Mean-Square (RMS) velocity fluctuations in Fig. 15.

While there is very good agreement between the present results and those from Abe et al. [28] for the averaged velocity profile and Reynolds stress, some deviations in RMS velocity profiles are visible in the present DUGKS simulation. One of the reasons for this could be the use of a coarse mesh in the channel center in both the spanwise and streamwise directions, leading to a higher numerical dissipation. The same observations were shown in Bo et al. [22].

Overall, compared to the results from Abe et al. [28], present DUGKS results obtained using GPU devices indicate that the GPU parallelization is correct and further validate the applicability of CUDA GPU devices for turbulence simulations using mesoscopic methods such as DUGKS.

5. Performance of the GPU implementation

Several GPUs were used for testing purposes, and their main properties are summarized in Table 4. GeForce 3060, GeForce 3090 and GeForce 4090 are consumer-grade GPU devices based on the “Ampere” (for GF3060 and GF3090) and Ada Lovelace (for GF4090) architectures and are used for gaming purposes. “Turing” architecture based device - RTX Quadro 6000 is a workstation-level GPU marketed mainly for professional visualization purposes. Both GeForce and Quadro families of GPUs have reduced double-precision (FP64) floating point operation speed, which is defined as fractions of single-precision (FP32) floating point operation speed. In the case of GeForce 3060, 3090 and 4090

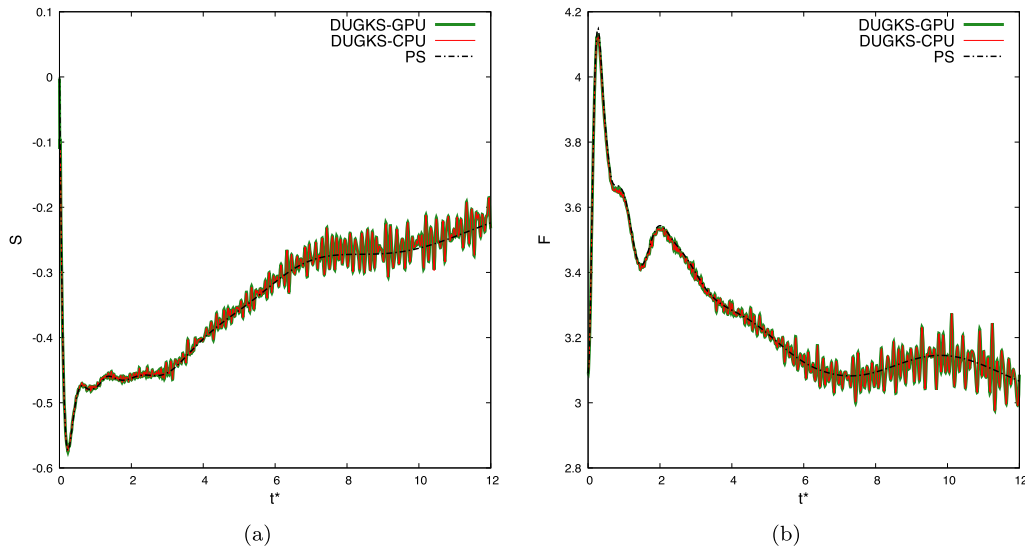


Fig. 11. Time evolution of a) the skewness and b) the flatness of velocity derivative for DHIT case with uniform mesh with a resolution of 256^3 .

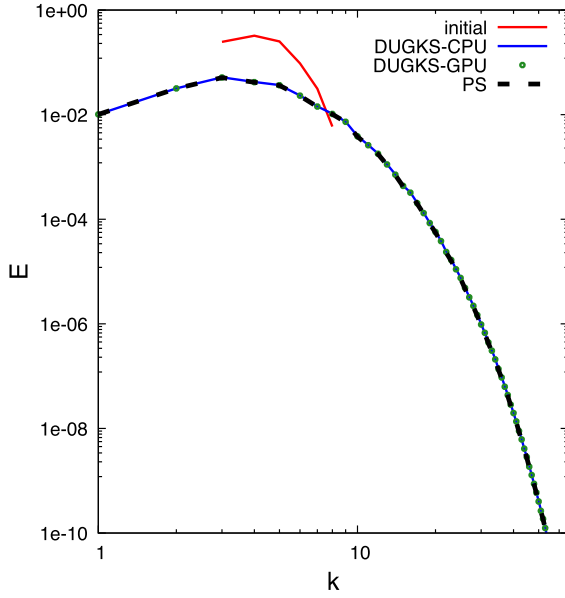


Fig. 12. Energy spectra at $t = 1$ for DHIT case with uniform mesh with a resolution of 256^3 . The data for “Initial” corresponds to the initial given energy spectrum.

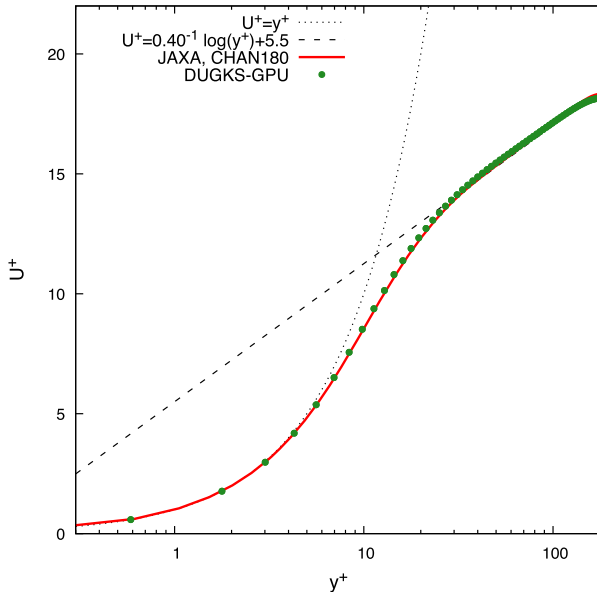


Fig. 13. The velocity profile in logarithmic scale for turbulent channel flow case with nonuniform mesh with the resolution of 128^3 . In the following plots, the data for “JAXA, CHAN180” is taken from the DNS database available at <https://jaxa-dns-database.jaxa.jp/channelflow.html> [28].

Table 4

Properties of the GPUs considered in this work.

Model	GF3060	GF3090	GF4090	RTX 6000	V100	A100
# of SMs	28	82	128	72	80	108
# of SPs per SM	128	128	128	64	64	64
Shared memory (KB)	128	128	128	64	128	192
Global memory (GB)	12	24	24	24	16	80
Memory bus (bit)	192	384	384	384	4096	5120
Bandwidth (GB/s)	360	936.2	1008	672	897	2039
SP freq. (MHz)	1320	1395	2235	1440	1245	1275
Mem. freq. (MHz)	1875	1219	1313	1750	876	1593

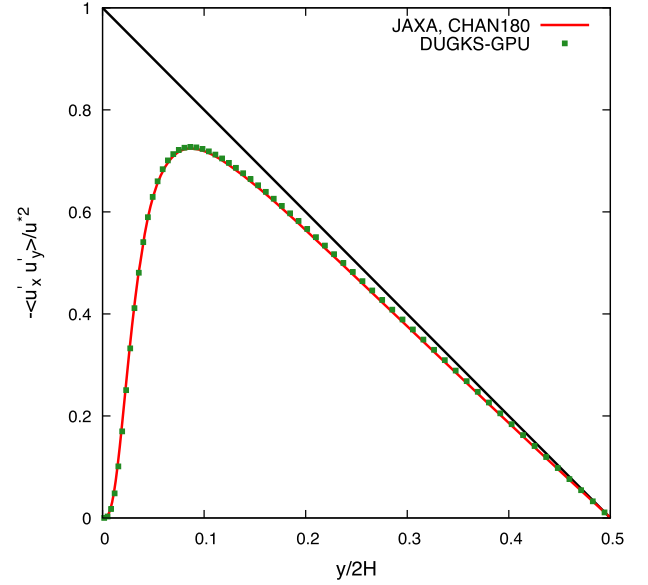


Fig. 14. Reynolds stress profile for turbulent channel flow case with nonuniform mesh with the resolution of 128^3 .

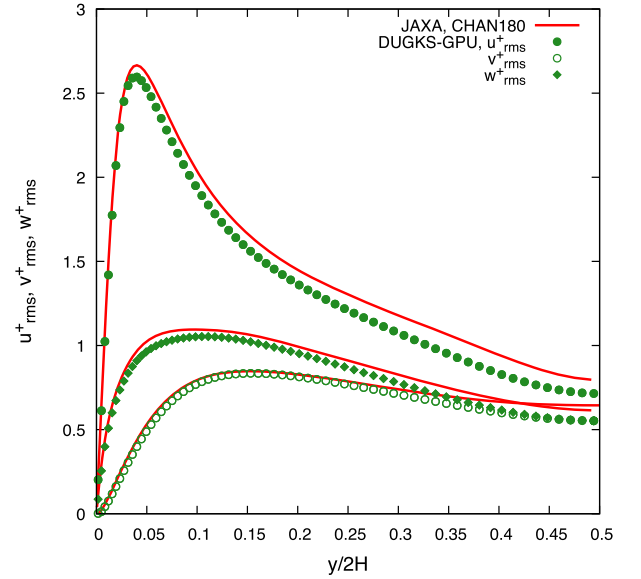


Fig. 15. RMS velocity profile for turbulent channel flow case with nonuniform mesh with the resolution of 128^3 .

this fraction is $1/64$, that is FP64 performance is 64 times slower than FP32 performance, and $1/32$ in the case of Quadro 6000 card. In contrast to consumer-grade GeForce and Quadro devices, Tesla V100 is a computation-related device with FP64 performance to FP32 performance ratio of $1/2$ and specially targeted for high-performance computations in multi-GPU and multi-node configurations.

Several works recently were devoted to performance analysis of GPU implementation of the GKS and DUGKS methods. DUGKS implementation on GPU achieved 338x speedup on V100 GPU over a single CPU thread in [21]. GKS implementation on GPU achieved approximately 11x speedup over 32 CPU cores in [29], while [30] achieved 2.8x speedup over 96 CPU cores in their iterative GKS approach. The Microhh project [10] achieved 1.59x speedup over 32 CPU cores in a single GPU simulation. While Microhh is a traditional CFD solver, its performance results still provide an overall comparison data from the usage of GPU and CPU platforms.

Table 5

Average runtime (in seconds) of 1000 timesteps for the Taylor-Green vortex flow case. Shown results were averaged over three successive runs.

Mesh	32CPU	GF3060	GF3090	GF4090	RTX6000	V100	A100
128 ³	44.48	20.53	8.34	6.32	7.26	7.25	4.51
256 ³	363.81	133.01	50.10	50.26	57.93	55.20	36.28
384 ³	1263.28	293.86	165.29	168.99	196.07	185.29	124.76

Table 6

Speedup over 32-core CPU for the Taylor-Green vortex flow case.

Mesh	GF3060	GF3090	GF4090	RTX6000	V100	A100
128 ³	2.17	5.33	7.04	6.13	6.14	9.86
256 ³	2.74	7.26	7.24	6.28	6.60	10.03
384 ³	4.30	7.64	7.48	6.44	6.82	10.13

Table 7

Execution performance in MCUPS (Mega Cell Updates Per Second) is compared across various mesh sizes and GPUs.

Mesh	32CPU	GF3060	GF3090	GF4090	RTX6000	V100	A100
128 ³	47	102	252	332	289	289	465
256 ³	46	126	335	334	290	304	462
384 ³	45	193	343	335	289	306	454

Table 8

Approximate speedup over single CPU core. DUGKS1 corresponds to [31], DUGKS2 corresponds to [20], microhh corresponds to [10], GKS corresponds to [29], IGKS corresponds to [30] and Present indicates results based on 384³ mesh on GF3090 device.

Present	DUGKS1	DUGKS2	Microhh	GKS	IGKS
245	338	29.36	51	88	269

Performance results from the current GPU implementation are shown in Table 5, where the runtimes for the Taylor-Green vortex flow problem were reported for the mesh resolutions of 128³, 256³, and 384³ in a single-precision mode. The interpretation of the same performance results in terms of Mega-Cell-Updates-Per-Second (MCUPS) is shown in Table 7. The CPU configuration used in these benchmark cases corresponds to the nodes of TaiYi supercomputer, with the properties listed in Table 9. Table 6 contains corresponding data for speedup on different GPUs. It is clear that the highest speedup value is achieved on the GeForce 3090 device, and the lowest speedup on the GeForce 3060 device. Both Quadro RTX 6000 and Tesla V100 attain lower performance which is explained by the lower memory bandwidth of these devices compared to the GeForce 3090. The active memory correction feature - ECC - additionally slows down memory transfers on V100. This observation indicates that DUGKS is a memory-bound algorithm on GPUs. Overall, attained speedup results are comparable to those in a recently published literature, and Table 8 shows approximate speedups of different methods relative to a single CPU core. Much lower speedup results from the Microhh project indicate that explicit time-marching algorithms, such as GKS or DUGKS, benefit more from the GPU parallel architecture.

Additionally, the effect of precision mode on runtime was studied for the case of turbulent channel flow, and related timings are shown in Table 10. Simulation parameters and non-uniform mesh configuration were the same as in section 4.3. For all GPUs under consideration, approximately the same performance ratio between single- and double-precision runs were achieved. Interestingly, the average value of this ratio is 2.53, which is relatively low compared to the declared ratio of 64 for GeForce 3090 and 32 for Quadro RTX 6000. This result, again, indicates that DUGKS on GPUs is a memory-bound method rather than compute-bound. To support these findings, the roofline model is shown

Table 9

The introduced interpolation approach reduces overall runtime by 21% relative to the same numerical simulation using gradient-based flux calculation on average.

Platform	MacBook Pro 15	Lenovo P320	TaiYi
OS	MacOS Big Sur	Debian 12	CentOS 7.5
CPU	Core i7 4770HQ	Core i7 7700	Xeon 6148
RAM	DDR3 1600	DDR4 2400	DDR4 2666
# of cores	4	4	32
Mesh	128 ³	192 ³	256 ³
Runtime reduction	17.5%	17.4%	27.4%

Table 10

Performance comparison of a single-precision and double-precision mode simulations applied to turbulent channel flow, with non-uniform mesh with a resolution of 128³.

Precision	GF3060	GF3090	GF4090	RTX6000	V100	A100
single	45.50	17.51	13.36	24.01	18.17	10.43
double	116.42	45.05	32.63	66.36	47.10	23.71
ratio	2.56	2.57	2.44	2.76	2.59	2.27

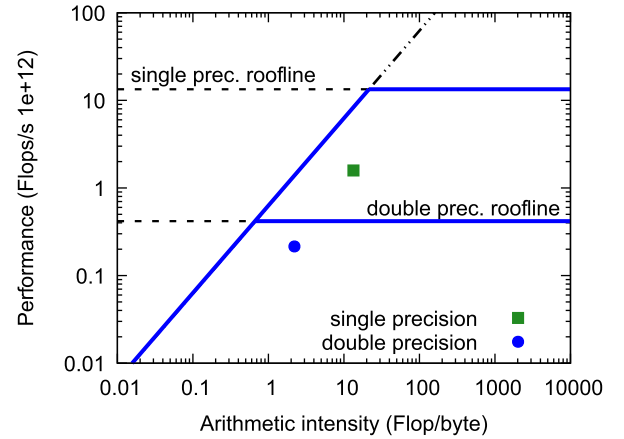


Fig. 16. The roofline model measured for Quadro RTX 6000 GPU. Two horizontal lines indicate roofline in a single and double precision mode.

in Fig. 16, from which it is evident that the present code is a memory bound application in single-precision mode.

The results of the strong-scaling tests from the multi-GPU version of the code are shown in Fig. 17, where superlinear speedup can be observed. Several reasons account for this superlinear speedup. First, all the GPUs used in this test are located on a single node with a high-bandwidth (600GB/s) NVLink GPU-GPU interconnect, which facilitates relatively fast inter-GPU communication. Second, performance data from Table 7 indicates that a smaller workload on the A100 GPU results in slightly better performance, thereby contributing to a better speedup in the strong scaling benchmark. Finally, the breakdown of communication-to-computation time in Fig. 18 shows a higher communication-to-computation ratio for the single GPU case compared to multiple GPU cases, effectively making the multi-GPU configuration more efficient in terms of communication-to-computation ratio.

6. Summary and outlook

This paper presents results from implementing DUGKS on a GPU system for laminar and turbulent flows. Two improvements were made to the implementation. First, the memory usage was significantly reduced compared to our previous CPU realization [22]. Second, a more-compact trilinear interpolation was proposed to interpolate the distribution functions for the flux computation. Code validation was performed on a Taylor-Green vortex decay, decaying homogeneous isotropic tur-

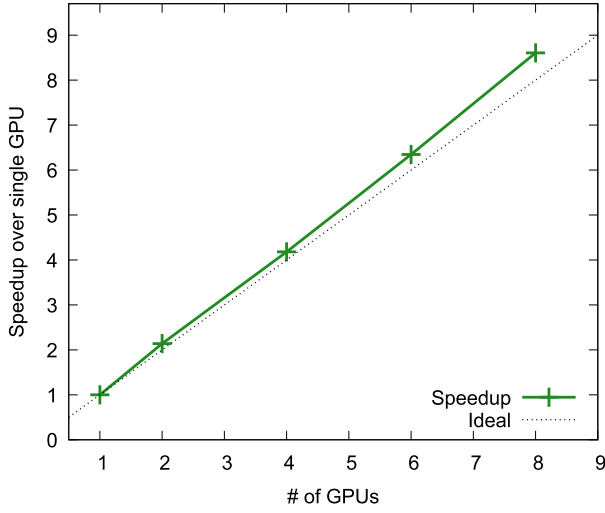


Fig. 17. Speedup of multi-GPU version on a fixed mesh of size $384 \times 384 \times 768$. The tests were performed on a multi-GPU node equipped with eight NVIDIA A100 GPUs with 40GB of memory and NVLink-600GB/s GPU interconnect was used.

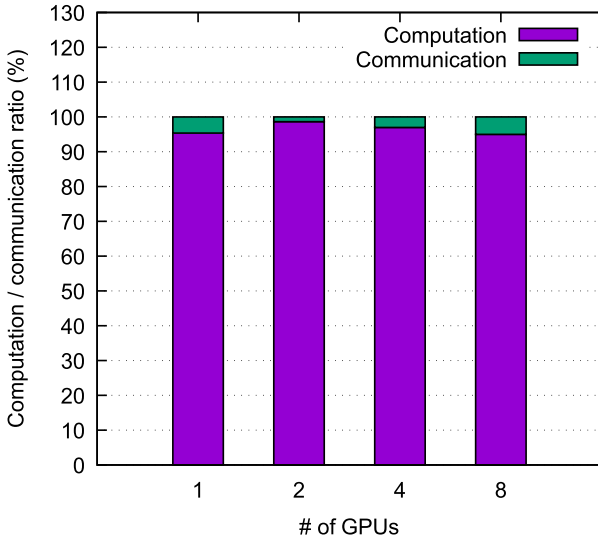


Fig. 18. The computation to communication ratio for different number of GPUs for a single node equipped with eight NVIDIA A100 GPUs with NVLink GPU interconnect.

bulence and turbulent channel flow. The Taylor-Green vortex flow case shows improvements for low and high-order statistics by using the trilinear interpolation, compared to the original interpolation used in DUGKS [22].

A single GPU run shows up to 7.64x speedup over 32 CPU core MPI run, which indicates overall good parallelization performance on a GPU. Given that modern GPUs contain a significant amount of main memory and several layers of it (global memory, shared memory, registers, caches), more computationally demanding problems can be addressed on GPU architectures. The trilinear interpolation procedure for flux calculation shows a reduction of the runtime of around 21% for three-dimensional problems on CPU systems, relative to the original gradient-based interpolation.

From this study, it can be concluded that GPUs are appealing tools for an explicit mesoscopic method such as DUGKS. The application of GPU-accelerated DUGKS code for the small to medium-scale study of laminar and turbulent flows can be efficiently performed, even on consumer-grade GPUs. The source code of the developed simulation

tool is released under the GPL license, allowing researchers to explore the kinetic method DUGKS for various laminar and turbulent flows.

Extensions of the current work include simulation of turbulent flows laden with solid objects, compressible flows and implementation on a multi-node GPU clusters with hundreds of GPUs. Vendor-independent GPU implementation using, for example, OpenCL standard [32] or Intel's oneAPI [33] is another option for an extension along with less utilized features such as the usage of mixed precision arithmetic and specialized matrix operation blocks.

CRedit authorship contribution statement

Kairzhan Karzhaubayev: Writing – original draft, Validation, Software, Methodology. **Lian-Ping Wang:** Writing – review & editing, Supervision, Software, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Dauren Zhakebayev:** Validation, Software, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work has been supported by the National Natural Science Foundation of China (NSFC award numbers T2250710183, U2241269, 91852205 & 11961131006), NSFC Basic Science Center Program (Award number 11988102), Guangdong Provincial Key Laboratory of Turbulence Research and Applications (2023B1212060001), Guangdong-Hong Kong-Macao Joint Laboratory for Data-Driven Fluid Mechanics and Engineering Applications (2020B1212030001), and Shenzhen Science & Technology Program (Grant No. KQTD20180411143441009), Ministry of Science and Higher Education of the Republic of Kazakhstan, grant No. AP19680366. Computing resources are provided by the Center for Computational Science and Engineering of Southern University of Science and Technology.

Appendix A. Taylor analysis of the interpolation scheme

Since the D3Q19 discrete velocity model used in this work contains no diagonal elements (such as $\{\pm 1, \pm 1, \pm 1\}$), we apply analysis to the two-dimensional case, with the particular discrete velocity direction $\xi_\alpha = \{1, -1\}$ (Fig. A.19).

To simplify the analysis further, it is assumed that the grid is uniform and $\Delta x = \Delta y = \Delta$, then, new quantities h, δ are defined as $h = 0.5\Delta t$ and $\delta = 0.5\Delta t c$. The single scalar value of the discrete distribution function f_α is denoted as g .

First, the exact value of $g(x_b - \delta, y_b + \delta)$ is found from the Taylor expansion around the node (x_b, y_b) :

$$g(x_b - \delta, y_b + \delta) = g(x_b, y_b) - \delta g_x + \delta g_y + \frac{1}{2} \delta^2 g_{xx} - \delta^2 g_{xy} + \frac{1}{2} \delta^2 g_{yy} + O(\delta^3). \quad (\text{A.1})$$

The interpolation approach estimates the value of $g(x_b - \delta, y_b + \delta)$ as

$$g(x_b - \delta, y_b + \delta) \approx (0.5 + \frac{\delta}{\Delta})(1 - \frac{\delta}{\Delta})g_{i,j} + (0.5 - \frac{\delta}{\Delta})(1 - \frac{\delta}{\Delta})g_{i+1,j} + (0.5 + \frac{\delta}{\Delta})(\frac{\delta}{\Delta})g_{i,j+1} + (0.5 - \frac{\delta}{\Delta})(\frac{\delta}{\Delta})g_{i+1,j+1}. \quad (\text{A.2})$$

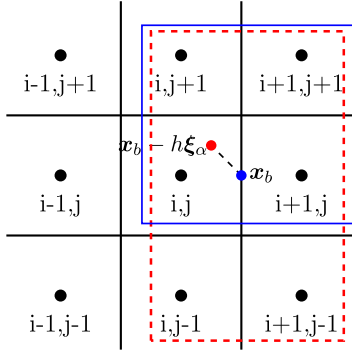


Fig. A.19. Comparison of stencils used in computing f_α , when $\xi_\alpha = (1, -1)$, for simplicity, shown for the two-dimensional configuration. The red dashed rectangle contains nodes used in gradient-based approach, while thin blue rectangle contains nodes which are used in trilinear interpolation based approach.

Further replacing the values of g at the nodes (i, j) , $(i + 1, j)$, $(i, j + 1)$, $(i + 1, j + 1)$ with their respective Taylor expansions around the node (x_b, y_b) results in

$$\begin{aligned} g(x_b - \delta, y_b + \delta) &= (0.5 + \frac{\delta}{\Delta})(1 - \frac{\delta}{\Delta}) \left[g(x_b, y_b) - \frac{\Delta}{2} g_x + \frac{\Delta^2}{8} g_{xx} + O(\Delta^3) \right] + \\ & (0.5 - \frac{\delta}{\Delta})(1 - \frac{\delta}{\Delta}) \left[g(x_b, y_b) + \frac{\Delta}{2} g_x + \frac{\Delta^2}{8} g_{xx} + O(\Delta^3) \right] + \\ & (0.5 + \frac{\delta}{\Delta})(\frac{\delta}{\Delta}) \left[g(x_b, y_b) - \frac{\Delta}{2} g_x + \Delta g_y + \frac{\Delta^2}{8} g_{xx} - \frac{\Delta^2}{2} g_{xy} + \right. \\ & \quad \left. \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right] + \\ & (0.5 - \frac{\delta}{\Delta})(\frac{\delta}{\Delta}) \left[g(x_b, y_b) + \frac{\Delta}{2} g_x + \Delta g_y + \frac{\Delta^2}{8} g_{xx} + \frac{\Delta^2}{2} g_{xy} + \right. \\ & \quad \left. \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right], \end{aligned} \quad (\text{A.3})$$

which, after rearranging the terms, produces the following final expression

$$\begin{aligned} g(x_b - \delta, y_b + \delta) &= g(x_b, y_b) + (-\delta)g_x + (\delta)g_y \\ &+ \left(\frac{\Delta^2}{8}\right)g_{xx} + (-\delta^2)g_{xy} + \left(\frac{\Delta\delta}{2}\right)g_{yy} + O(\delta^3, \delta^2\Delta, \delta\Delta^2, \Delta^3). \end{aligned} \quad (\text{A.4})$$

Similarly, the gradient-based estimation of $g(x_b - \delta, y_b + \delta)$ is

$$\begin{aligned} g(x_b - \delta, y_b + \delta) &\approx \left(\frac{1}{2} + \frac{\delta}{\Delta}\right) g_{i,j} + \left(\frac{1}{2} - \frac{\delta}{\Delta}\right) g_{i+1,j} + \frac{1}{4} \frac{\delta}{\Delta} g_{i,j+1} + \\ & \frac{1}{4} \frac{\delta}{\Delta} g_{i+1,j+1} - \frac{1}{4} \frac{\delta}{\Delta} g_{i,j-1} - \frac{1}{4} \frac{\delta}{\Delta} g_{i+1,j-1}, \end{aligned} \quad (\text{A.5})$$

where the values of g at the nodes (i, j) , $(i, j + 1)$, $(i, j - 1)$, $(i + 1, j)$, $(i + 1, j + 1)$, $(i + 1, j - 1)$ are replaced with their Taylor series approximations around the node (x_b, y_b) :

$$\begin{aligned} g(x_b - \delta, y_b + \delta) &= \\ & \left(\frac{1}{2} + \frac{\delta}{\Delta}\right) \left(g(x_b, y_b) - \frac{\Delta}{2} g_x + \frac{\Delta^2}{8} g_{xx} + O(\Delta^3) \right) + \\ & \left(\frac{1}{2} - \frac{\delta}{\Delta}\right) \left(g(x_b, y_b) + \frac{\Delta}{2} g_x + \frac{\Delta^2}{8} g_{xx} + O(\Delta^3) \right) + \\ & \frac{1}{4} \frac{\delta}{\Delta} \left(g(x_b, y_b) - \frac{\Delta}{2} g_x + \Delta g_y + \frac{\Delta^2}{8} g_{xx} - \frac{\Delta^2}{2} g_{xy} + \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right) + \\ & \frac{1}{4} \frac{\delta}{\Delta} \left(g(x_b, y_b) + \frac{\Delta}{2} g_x + \Delta g_y + \frac{\Delta^2}{8} g_{xx} + \frac{\Delta^2}{2} g_{xy} + \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right) + \\ & - \frac{1}{4} \frac{\delta}{\Delta} \left(g(x_b, y_b) - \frac{\Delta}{2} g_x - \Delta g_y + \frac{\Delta^2}{8} g_{xx} + \frac{\Delta^2}{2} g_{xy} + \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right) + \end{aligned}$$

$$- \frac{1}{4} \frac{\delta}{\Delta} \left(g(x_b, y_b) + \frac{\Delta}{2} g_x - \Delta g_y + \frac{\Delta^2}{8} g_{xx} - \frac{\Delta^2}{2} g_{xy} + \frac{\Delta^2}{2} g_{yy} + O(\Delta^3) \right) \quad (\text{A.6})$$

Again, after rearranging the similar terms, the final form is

$$\begin{aligned} g(x_b - \delta, y_b + \delta) &= g(x_b, y_b) + (-\delta)g_x + (\delta)g_y + \left(\frac{\Delta^2}{8}\right)g_{xx} + (0)g_{xy} + (0)g_{yy} \\ &+ O(\delta^3, \delta^2\Delta, \delta\Delta^2, \Delta^3). \end{aligned} \quad (\text{A.7})$$

In summary, the final Taylor series expansions of the gradient-based and the interpolation-based approaches with the exact value are:

for gradient-based approach:

$$\begin{aligned} g(x_b, y_b) &+ (-\delta)g_x + (\delta)g_y + \frac{\Delta^2}{8} g_{xx} + 0 \cdot g_{xy} + 0 \cdot g_{yy} \\ &+ O(\delta^3, \delta^2\Delta, \delta\Delta^2, \Delta^3), \end{aligned} \quad (\text{A.8})$$

for interpolation-based:

$$\begin{aligned} g(x_b, y_b) &+ (-\delta)g_x + (\delta)g_y + \frac{\Delta^2}{8} g_{xx} + (-\delta^2)g_{xy} + \frac{\Delta\delta}{2} g_{yy} \\ &+ O(\delta^3, \delta^2\Delta, \delta\Delta^2, \Delta^3), \end{aligned} \quad (\text{A.9})$$

and the exact expression:

$$g(x_b, y_b) + (-\delta)g_x + (\delta)g_y + \frac{\delta^2}{2} g_{xx} + (-\delta^2)g_{xy} + \frac{\delta^2}{2} g_{yy} + O(\delta^3). \quad (\text{A.10})$$

The comparison of the gradient-based (A.8) and interpolation-based (A.9) approximations to the exact expression (A.10) provide several valuable observations. First, both gradient- and interpolation-based approaches are of second-order accuracy. Second, the correction term related to the derivative g_{xx} is the same in both approximations, while the correction related to g_{xy} term is exact in the interpolation-based approach. Furthermore, in a typical simulation, g_{yy} correction term is better approximated using the interpolation-based approach, compared to the gradient-based one, in which g_{yy} correction term is always zero.

References

- [1] S. Tomov, J. Dongarra, M. Baboulin, Towards dense linear algebra for hybrid GPU accelerated manycore systems, *Parallel Comput.* 36 (5–6) (2010) 232–240.
- [2] E. Lindholm, J. Nickolls, S. Oberman, J. Montrym, NVIDIA Tesla: a unified graphics and computing architecture, *IEEE MICRO* 28 (2) (2008) 39–55, <https://doi.org/10.1109/MM.2008.31>.
- [3] S. Succi, *The Lattice Boltzmann Equation: for Fluid Dynamics and Beyond*, Oxford University Press, 2001.
- [4] K. Xu, J.-C. Huang, A unified gas-kinetic scheme for continuum and rarefied flows, *J. Comput. Phys.* 229 (20) (2010) 7747–7764.
- [5] Z. Guo, K. Xu, R. Wang, Discrete unified gas kinetic scheme for all Knudsen number flows: low-speed isothermal case, *Phys. Rev. E* 88 (3) (2013) 033305.
- [6] C. Feichtinger, J. Habich, H. Köstler, U. Rüde, T. Aoki, Performance modeling and analysis of heterogeneous lattice Boltzmann simulations on CPU-GPU clusters, *Parallel Comput.* 46 (2015) 1–13, <https://doi.org/10.1016/j.parco.2014.12.003>, univIS-Import:2015-03-09:Pub, 2014.tech.IMMD.lsinfs.perfor.79.
- [7] J. Tölke, Implementation of a lattice Boltzmann kernel using the compute unified device architecture developed by NVIDIA, *Comput. Vis. Sci.* 13 (1) (2010) 29–39.
- [8] C. Obrecht, F. Kuznik, B. Tourancheau, J.-J. Roux, A new approach to the lattice Boltzmann method for graphics processing units, *Comput. Math. Appl.* 61 (12) (2011) 3628–3638.
- [9] J. Habich, C. Feichtinger, H. Köstler, G. Hager, G. Wellein, Performance engineering for the lattice Boltzmann method on GPGPUs: architectural requirements and performance results, *Comput. Fluids* 80 (2013) 276–282.
- [10] C.C. Van Heerwaarden, B.J. Van Stratum, T. Heus, J.A. Gibbs, E. Fedorovich, J.P. Mellado, Microhh 1.0: a computational fluid dynamics code for direct numerical simulation and large-eddy simulation of atmospheric boundary layer flows, *Geosci. Model Dev.* 10 (8) (2017) 3145–3165.
- [11] P. Bailey, J. Myre, D. Walsh, D. Lilja, M. Saar, Accelerating lattice Boltzmann fluid flow simulations using graphics processors, in: 2009 International Conference on Parallel Processing, 2009.
- [12] P. Wang, L.-P. Wang, Z. Guo, Comparison of the lattice Boltzmann equation and discrete unified gas-kinetic scheme methods for direct numerical simulation of decaying turbulent flows, *Phys. Rev. E* 94 (4) (2016) 043304.
- [13] T. Chen, X. Wen, L.-P. Wang, Z. Guo, J. Wang, S. Chen, D.B. Zhakebayev, Simulation of three-dimensional forced compressible isotropic turbulence by a redesigned discrete unified gas kinetic scheme, *Phys. Fluids* 34 (2) (2022) 025106.

- [14] C. Li, L.-P. Wang, An immersed boundary-discrete unified gas kinetic scheme for simulating natural convection involving curved surfaces, *Int. J. Heat Mass Transf.* 126 (2018) 1059–1070.
- [15] X. Wen, L.-P. Wang, Z. Guo, D.B. Zhakebayev, Laminar to turbulent flow transition inside the boundary layer adjacent to isothermal wall of natural convection flow in a cubical cavity, *Int. J. Heat Mass Transf.* 167 (2021) 120822.
- [16] S. Tao, H. Zhang, Z. Guo, L.-P. Wang, A combined immersed boundary and discrete unified gas kinetic scheme for particle–fluid flows, *J. Comput. Phys.* 375 (2018) 498–518.
- [17] K. Karzhaubayev, L.-P. Wang, C. Peng, D. Zhakebayev, An immersed boundary method-discrete unified gas kinetic scheme simulation of particle-laden turbulent channel flow on a nonuniform orthogonal mesh, *Int. J. Numer. Methods Fluids* 96 (2024) 318–335, <https://doi.org/10.1002/fld.5246>.
- [18] Z. Guo, K. Xu, Discrete unified gas kinetic scheme for multiscale heat transfer based on the phonon Boltzmann transport equation, *Int. J. Heat Mass Transf.* 102 (2016) 944–958, <https://doi.org/10.1016/j.ijheatmasstransfer.2016.06.088>.
- [19] Z. Guo, K. Xu, Progress of discrete unified gas-kinetic scheme for multiscale flows, *Adv. Aerodyn.* 3 (1) (2021) 1–42.
- [20] J. Shang, Z. Chai, X. Chen, B. Shi, Discrete unified gas kinetic scheme for incompressible Navier-Stokes equations, *Comput. Math. Appl.* 97 (2021) 45–60.
- [21] P. Liu, C. Huang, Z. Guo, GPU implementation of the discrete unified gas kinetic scheme for low-speed isothermal flows, *Comput. Phys. Commun.* 294 (2024) 108908.
- [22] Y. Bo, P. Wang, Z. Guo, L.-P. Wang, DUGKS simulations of three-dimensional Taylor–green vortex flow and turbulent channel flow, *Comput. Fluids* 155 (2017) 9–21.
- [23] L. Zhu, S. Chen, Z. Guo, dugksfoam: an open source openfoam solver for the Boltzmann model equation, *Comput. Phys. Commun.* 213 (2013) 155–164.
- [24] C. Wu, B. Shi, Z. Chai, P. Wang, Discrete unified gas kinetic scheme with a force term for incompressible fluid flows, *Comput. Math. Appl.* 71 (12) (2016) 2608–2629.
- [25] R. Mei, L.-S. Luo, P. Lallemand, D. d’Humières, Consistent initial conditions for lattice Boltzmann simulations, *Comput. Fluids* 35 (8–9) (2006) 855–862.
- [26] K. Karzhaubayev, L.-P. Wang, D. Zhakebayev, An efficient parallel spectral code for 3D periodic flow simulations, *SoftwareX* 20 (2022) 101244.
- [27] R.S. Rogallo, Numerical Experiments in Homogeneous Turbulence, vol. 81315, National Aeronautics and Space Administration, 1981.
- [28] H. Abe, H. Kawamura, Y. Matsuo, Direct numerical simulation of a fully developed turbulent channel flow with respect to the Reynolds number dependence, *J. Fluids Eng.* 123 (2) (2001) 382–393, <https://doi.org/10.1115/1.1366680>, arXiv:https://asmedigitalcollection.asme.org/fluidsengineering/article-pdf/123/2/382/5590817/382_1.pdf.
- [29] Y. Wang, L. Pan, Three-dimensional discontinuous Galerkin based high-order gas-kinetic scheme and GPU implementation, *Comput. Fluids* 242 (2022) 105510, <https://doi.org/10.1016/j.compfluid.2022.105510>.
- [30] L. Zhu, P. Wang, S. Chen, Z. Guo, Y. Zhang, GPU acceleration of an iterative scheme for gas-kinetic model equations with memory reduction techniques, *Comput. Phys. Commun.* 245 (2019) 106861.
- [31] P. Liu, The 18th International Conference for Mesoscopic Methods in Engineering and Science, La Rochelle, France, 2022.
- [32] J.E. Stone, D. Gohara, G. Shi, OpenCL: a parallel programming standard for heterogeneous computing systems, *Comput. Sci. Eng.* 12 (3) (2010) 66.
- [33] S. Christgau, T. Steinke, Porting a legacy CUDA stencil code to oneAPI, in: 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), IEEE, 2020, pp. 359–367.